

✓ Assignment No. 5

Apache Spark DataFrame Operations using PySpark

Objective

The objective of this assignment is to understand Apache Spark fundamentals and perform data cleaning, transformation, filtering, aggregation, grouping, schema modification, and build a complete data processing pipeline using Spark DataFrames.

Tools Used

- Google Colab
- Apache Spark (PySpark)
- Python
- Titanic Dataset (CSV)

✓ Install PySpark

PySpark is the Python API for Apache Spark.

Google Colab does not include PySpark by default, so we need to install it.

```
# =====  
# Install PySpark  
# =====  
  
!pip install pyspark -q  
  
print("PySpark Installed Successfully.")  
  
PySpark Installed Successfully.
```

✓ Import Required Libraries

In this step, we import all the required libraries to work with Spark DataFrames.

```
# =====  
# Import Required Libraries  
# =====
```

```
# Import Spark Session
from pyspark.sql import SparkSession

# Import commonly used SQL functions
from pyspark.sql.functions import *

# Import Spark Data Types
from pyspark.sql.types import *

print("Libraries Imported Successfully.")
```

Libraries Imported Successfully.

✓ Create Spark Session

A Spark Session is the entry point for working with Apache Spark.

Without a Spark Session, we cannot create or process Spark DataFrames.

```
# =====
# Create Spark Session
# =====

spark = SparkSession.builder \
    .appName("Titanic Spark Assignment") \
    .master("local[*]") \
    .getOrCreate()

print("Spark Session Created Successfully.")
```

Spark Session Created Successfully.

✓ Upload Titanic Dataset

Upload the Titanic-Dataset.csv file downloaded from Kaggle.

After running the next cell, click **Choose Files** and select:

Titanic-Dataset.csv

```
# =====
# Upload Titanic Dataset
# =====

from google.colab import files

uploaded = files.upload()

print("Dataset Uploaded Successfully.")
```

Choose files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving Titanic-Dataset.csv to Titanic-Dataset.csv

Dataset Unloaded Successfully

✓ Load CSV File into Spark DataFrame

In this step, we load the uploaded Titanic dataset into a Spark DataFrame.

The options used are:

- `header=True` → Uses the first row as column names.
- `inferSchema=True` → Automatically detects data types.

```
# =====
# Read Titanic Dataset
# =====

df = spark.read.csv(
    "Titanic-Dataset.csv",
    header=True,
    inferSchema=True
)

print("Dataset Loaded Successfully.")
```

Dataset Loaded Successfully.

✓ Display Dataset

The `show()` function displays the first few rows of the DataFrame.

```
# =====
# Display First 10 Records
# =====

df.show(10, truncate=False)
```

PassengerId	Survived	Pclass	Name
1	0	3	Braund, Mr. Owen Harris
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)
3	1	3	Heikkinen, Miss. Laina
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)
5	0	3	Allen, Mr. William Henry
6	0	3	Moran, Mr. James
7	0	1	McCarthy, Mr. Timothy J
8	0	3	Palsson, Master. Gosta Leonard
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)

```
|10      |1      |2      |Nasser, Mrs. Nicholas (Adele Achem)
+-----+-----+-----+-----+
only showing top 10 rows
```

Display Column Names

This helps us understand the available features in the dataset.

```
# =====
# Print Column Names
# =====

print(df.columns)

['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'T
```

Generate Summary Statistics

The describe() function provides statistical information for numeric columns such as:

- Count
- Mean
- Standard Deviation
- Minimum
- Maximum

```
# =====
# Display Summary Statistics
# =====

df.describe().show()
```

summary	PassengerId	Survived	Pclass
count	891	891	891
mean	446.0	0.3838383838383838	2.308641975308642
stddev	257.3538420152301	0.48659245426485753	0.8360712409770491
min	1	0	1
max	891	1	3

Data Cleaning

Before analyzing the data, we need to check its quality.

In this section, we will:

- Check for duplicate records

- Check missing values
- Handle null values
- Improve the dataset for analysis

```
# =====
# Check Total Rows Before Removing Duplicates
# =====

print("Total Rows Before Removing Duplicates :", df.count())
```

Total Rows Before Removing Duplicates : 891

✓ Remove Duplicate Records

Duplicate records increase storage and may produce incorrect analysis.

The `dropDuplicates()` function removes duplicate rows.

```
# =====
# Remove Duplicate Rows
# =====

df = df.dropDuplicates()

print("Total Rows After Removing Duplicates :", df.count())
```

Total Rows After Removing Duplicates : 891

✓ Check Missing Values

Missing values reduce data quality.

We will count null values in every column.

```
# =====
# Count Missing Values in Every Column
# =====

from pyspark.sql.functions import col, when, count

df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
]).show()
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|          0 |          0 |          0 | 0 | 0 | 177 |          0 |          0 |          0 |          0 | 687 |          2
```

+-----+-----+-----+-----+-----+-----+-----+-----+

- Fill Missing Age Values

Instead of deleting records, we replace missing Age values with the average age.

```
# =====
# Calculate Average Age
# =====

average_age = df.select(avg("Age")).collect()[0][0]

print("Average Age :", average_age)
```

Average Age : 29.69911764705882

```
# =====
# Replace Null Age with Average Age
# =====

df = df.fillna({
    "Age": average_age
})

print("Missing Age Values Filled Successfully")
```

Missing Age Values Filled Successfully

- ✦ Fill Missing Embarked Values

Replace missing Embarked values with "Unknown".

```
# =====  
# Fill Missing Embarked Values  
# =====  
  
df = df.fillna({  
    "Embarked": "Unknown"  
)  
  
print("Missing Embarked Values Filled")
```

Missing Embarked Values Filled

- ✓ Fill Missing Cabin Values

The Cabin column contains many missing values.

We replace them with "Not Assigned".

```
# =====
# Fill Missing Cabin Values
# =====

df = df.fillna({
    "Cabin": "Not Assigned"
})

print("Cabin Values Updated")
```

Cabin Values Updated

✓ Verify Missing Values Again

After cleaning, we verify that all missing values have been handled.

```
# =====
# Verify Missing Values
# =====

df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
]).show()
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|           0 |         0 |        0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

✓ Rename Column

Renaming columns makes the dataset easier to understand.

```
# =====
# Rename Fare Column
# =====

df = df.withColumnRenamed(
    "Fare",
    "TicketFare"
)

print("Column Renamed Successfully")
```

Column Renamed Successfully

✓ Verify Column Names

```
# =====
# Display Updated Column Names
# =====

print(df.columns)

['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']
```

✓ Change Data Type

Convert the TicketFare column into Double data type.

```
# =====
# Convert TicketFare into Double
# =====

df = df.withColumn(
    "TicketFare",
    col("TicketFare").cast("double")
)

print("Data Type Converted Successfully")
```

Data Type Converted Successfully

```
# =====
# Display Updated Schema
# =====

df.printSchema()
```

```
root
|-- PassengerId: integer (nullable = true)
|-- Survived: integer (nullable = true)
|-- Pclass: integer (nullable = true)
|-- Name: string (nullable = true)
|-- Sex: string (nullable = true)
|-- Age: double (nullable = false)
|-- SibSp: integer (nullable = true)
|-- Parch: integer (nullable = true)
|-- Ticket: string (nullable = true)
|-- TicketFare: double (nullable = true)
|-- Cabin: string (nullable = false)
|-- Embarked: string (nullable = false)
```

✓ Create New Column

Create a new column named FareWithTax.

Formula:

FareWithTax = TicketFare + 18% GST

```
# =====
# Create FareWithTax Column
# =====

df = df.withColumn(
    "FareWithTax",
    round(col("TicketFare") * 1.18, 2)
)

df.show(5)
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 2 | 1 | 1 | Cumings, Mrs. Joh... | female | 38.0 | 1 | 0 | PC 175 |
| 24 | 1 | 1 | Sloper, Mr. Willi... | male | 28.0 | 0 | 0 | 1137 |
| 680 | 1 | 1 | Cardeza, Mr. Thom... | male | 36.0 | 0 | 1 | PC 177 |
| 292 | 1 | 1 | Bishop, Mrs. Dick... | female | 19.0 | 1 | 0 | 119 |
| 330 | 1 | 1 | Hippach, Miss. Je... | female | 16.0 | 0 | 1 | 1113 |
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

✓ Display Cleaned Dataset

The dataset is now cleaned and transformed.

```
# =====
# Display Cleaned Dataset
# =====

df.show(10, truncate=False)
```

```
+-----+-----+-----+-----+
| PassengerId | Survived | Pclass | Name |
+-----+-----+-----+-----+
| 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Thayer) |
| 24 | 1 | 1 | Sloper, Mr. William Thompson |
| 680 | 1 | 1 | Cardeza, Mr. Thomas Drake Martinez |
| 292 | 1 | 1 | Bishop, Mrs. Dickinson H (Helen Walton) |
| 330 | 1 | 1 | Hippach, Miss. Jean Gertrude |
| 488 | 0 | 1 | Kent, Mr. Edward Austin |
| 890 | 1 | 1 | Behr, Mr. Karl Howell |
```

264	0	1	Harrison, Mr. William
394	1	1	Newell, Miss. Marjorie
270	1	1	Bissette, Miss. Amelia

only showing top 10 rows

✓ Data Filtering

Filtering allows us to retrieve only the records that satisfy a given condition.

In this section, we will filter passengers based on:

- Age
- Gender
- Passenger Class
- Ticket Fare
- Multiple Conditions

```
# =====
# Filter Passengers Between Age 20 and 40
# =====

age_filter = df.filter(
    (col("Age") >= 20) &
    (col("Age") <= 40)
)

print("Passengers Between Age 20 and 40")

age_filter.show(10)
```

Passengers Between Age 20 and 40

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
2	1	1	Cumings, Mrs. Joh...	female	38.0	1	0	PC 175
24	1	1	Sloper, Mr. Willi...	male	28.0	0	0	1137
680	1	1	Cardeza, Mr. Thom...	male	36.0	0	1	PC 177
890	1	1	Behr, Mr. Karl Ho...	male	26.0	0	0	1113
264	0	1	Harrison, Mr. Wil...	male	40.0	0	0	1120
394	1	1	Newell, Miss. Mar...	female	23.0	1	0	352
270	1	1	Bissette, Miss. A...	female	35.0	0	0	PC 177
370	1	1	Aubart, Mme. Leon...	female	24.0	0	0	PC 174
140	0	1	Giglio, Mr. Victor	male	24.0	0	0	PC 175
642	1	1	Sagesser, Mlle. Emma	female	24.0	0	0	PC 174

only showing top 10 rows

Filter Female Passengers

Display all female passengers.

```
# =====
# Filter Female Passengers
# =====

female_df = df.filter(col("Sex") == "female")

print("Female Passengers")

female_df.show(10)
```

Female Passengers

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ti
2	1	1	Cumings, Mrs. Joh...	female	38.0	1	0	PC 1
292	1	1	Bishop, Mrs. Dick...	female	19.0	1	0	1
330	1	1	Hippach, Miss. Je...	female	16.0	0	1	11
394	1	1	Newell, Miss. Mar...	female	23.0	1	0	3
270	1	1	Bissette, Miss. A...	female	35.0	0	0	PC 1
370	1	1	Aubart, Mme. Leon...	female	24.0	0	0	PC 1
642	1	1	Sagesser, Mlle. Emma	female	24.0	0	0	PC 1
311	1	1	Hays, Miss. Marga...	female	24.0	0	0	1
517	1	2	Lemore, Mrs. (Ame...	female	34.0	0	0	C.A. 3
342	1	1	Fortune, Miss. Al...	female	24.0	3	2	1

only showing top 10 rows

Filter First Class Passengers

Display passengers travelling in First Class.

```
# =====
# Filter First Class Passengers
# =====

first_class = df.filter(col("Pclass") == 1)

print("First Class Passengers")

first_class.show(10)
```

First Class Passengers

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Tick
2	1	1	Cumings, Mrs. Joh...	female	38.0	1	0	PC 175

24	1	1	Sloper, Mr. Willi...	male	28.0	0	0	1137
680	1	1	Cardeza, Mr. Thom...	male	36.0	0	1	PC 177
292	1	1	Bishop, Mrs. Dick...	female	19.0	1	0	119
330	1	1	Hippach, Miss. Je...	female	16.0	0	1	1113
488	0	1	Kent, Mr. Edward ...	male	58.0	0	0	117
890	1	1	Behr, Mr. Karl Ho...	male	26.0	0	0	1113
264	0	1	Harrison, Mr. Wil...	male	40.0	0	0	1120
394	1	1	Newell, Miss. Mar...	female	23.0	1	0	352
270	1	1	Bissette, Miss. A...	female	35.0	0	0	PC 177

only showing top 10 rows

✓ Filter High Fare Passengers

Display passengers whose Ticket Fare is greater than 100.

```
# =====
# Ticket Fare Greater Than 100
# =====

high_fare = df.filter(col("TicketFare") > 100)

print("Passengers Paying Fare Greater Than 100")

high_fare.show()
```

Passengers Paying Fare Greater Than 100

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
680	1	1	Cardeza, Mr. Thom...	male	36.0	0	1	PC 177
394	1	1	Newell, Miss. Mar...	female	23.0	1	0	352
270	1	1	Bissette, Miss. A...	female	35.0	0	0	PC 177
342	1	1	Fortune, Miss. Al...	female	24.0	3	2	199
803	1	1	Carter, Master. W...	male	11.0	1	2	1137
320	1	1	Spedden, Mrs. Fre...	female	40.0	1	1	169
582	1	1	Thayer, Mrs. John...	female	39.0	1	1	174
690	1	1	Madill, Miss. Geo...	female	15.0	0	1	241
545	0	1	Douglas, Mr. Walt...	male	50.0	1	0	PC 177
551	1	1	Thayer, Mr. John ...	male	17.0	0	2	174
216	1	1	Newell, Miss. Mad...	female	31.0	1	0	352
764	1	1	Carter, Mrs. Will...	female	36.0	1	2	1137
28	0	1	Fortune, Mr. Char...	male	19.0	3	2	199
333	0	1	Graham, Mr. Georg...	male	38.0	0	1	PC 175
610	1	1	Shutes, Miss. Eli...	female	40.0	0	0	PC 175
338	1	1	Burns, Miss. Eliz...	female	41.0	0	0	169
298	0	1	Allison, Miss. He...	female	2.0	1	2	1137
89	1	1	Fortune, Miss. Ma...	female	23.0	3	2	199
506	0	1	Penasco y Castell...	male	18.0	1	0	PC 177
436	1	1	Carter, Miss. Luc...	female	14.0	1	2	1137

only showing top 20 rows

✓ Apply Multiple Conditions

Display female passengers travelling in First Class.

```
# =====
# Multiple Filtering Conditions
# =====

result = df.filter(
    (col("Sex") == "female") &
    (col("Pclass") == 1)
)

print("Female Passengers in First Class")

result.show()
```

Female Passengers in First Class

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
2	1	1	Cumings, Mrs. Joh...	female	38.0	1	0	PC 175
292	1	1	Bishop, Mrs. Dick...	female	19.0	1	0	119
330	1	1	Hippach, Miss. Je...	female	16.0	0	1	1113
394	1	1	Newell, Miss. Mar...	female	23.0	1	0	352
270	1	1	Bissette, Miss. A...	female	35.0	0	0	PC 177
370	1	1	Aubart, Mme. Leon...	female	24.0	0	0	PC 174
642	1	1	Sagesser, Mlle. Emma	female	24.0	0	0	PC 174
311	1	1	Hays, Miss. Marga...	female	24.0	0	0	117
342	1	1	Fortune, Miss. Al...	female	24.0	3	2	199
357	1	1	Bowerman, Miss. E...	female	22.0	0	1	1135
53	1	1	Harper, Mrs. Henr...	female	49.0	1	0	PC 175
320	1	1	Spedden, Mrs. Fre...	female	40.0	1	1	169
582	1	1	Thayer, Mrs. John...	female	39.0	1	1	174
690	1	1	Madill, Miss. Geo...	female	15.0	0	1	241
195	1	1	Brown, Mrs. James...	female	44.0	0	0	PC 176
766	1	1	Hogeboom, Mrs. Jo...	female	51.0	1	0	135
413	1	1	Minahan, Miss. Da...	female	33.0	1	0	199
216	1	1	Newell, Miss. Mad...	female	31.0	1	0	352
764	1	1	Carter, Mrs. Will...	female	36.0	1	2	1137
782	1	1	Dick, Mrs. Albert...	female	17.0	1	0	174

only showing top 20 rows

✓ Step 16

Aggregation Functions

Aggregation functions summarize data.

Functions used:

- count()
- sum()
- avg()
- min()
- max()

```
# =====
# Total Number of Passengers
# =====

print("Total Passengers :", df.count())
```

Total Passengers : 891

✓ Average Ticket Fare

```
# =====
# Average Ticket Fare
# =====

df.select(
    avg("TicketFare").alias("Average Fare")
).show()
```

```
+-----+
|      Average Fare|
+-----+
|32.20420796857458|
+-----+
```

✓ Total Ticket Fare

```
# =====
# Total Ticket Fare
# =====

df.select(
    sum("TicketFare").alias("Total Fare")
).show()
```

```
+-----+
|      Total Fare|
+-----+
|28693.94929999995|
+-----+
```

+-----+

✓ Maximum Ticket Fare

```
# =====
# Maximum Ticket Fare
# =====

df.select(
    max("TicketFare").alias("Maximum Fare")
).show()
```

```
+-----+
|Maximum Fare|
+-----+
|      512.3292|
+-----+
```

✓ Minimum Ticket Fare

```
# =====
# Minimum Ticket Fare
# =====

df.select(
    min("TicketFare").alias("Minimum Fare")
).show()
```

```
+-----+
|Minimum Fare|
+-----+
|           0.0|
+-----+
```

✓ GroupBy Operations

GroupBy combines rows having the same values and performs aggregation.

We will group data by:

- Gender
- Passenger Class
- Embarked Port

```
# =====
# Count Passengers by Gender
# =====

gender_count = df.groupBy("Sex").count()

gender_count.show()
```

```
+-----+-----+
|  Sex | count |
+-----+-----+
| female | 314 |
|  male | 577 |
+-----+-----+
```

```
# =====
# Average Fare by Passenger Class
# =====

df.groupBy("Pclass") \
  .agg(
    avg("TicketFare").alias("Average Fare")
  ) \
  .show()
```

```
+-----+-----+
| Pclass | Average Fare |
+-----+-----+
| 1 | 84.15468749999998 |
| 3 | 13.675550101833 |
| 2 | 20.662183152173913 |
+-----+-----+
```

```
# =====
# Count Passengers by Embarked Port
# =====

df.groupBy("Embarked") \
  .count() \
  .show()
```

```
+-----+-----+
| Embarked | count |
+-----+-----+
| Q | 77 |
| Unknown | 2 |
| C | 168 |
| S | 644 |
+-----+-----+
```

```
# =====
# Average Age by Gender
# =====
```

```
df.groupBy("Sex") \
  .agg(
    avg("Age").alias("Average Age")
  ) \
  .show()
```

```
+-----+-----+
|  Sex|      Average Age|
+-----+-----+
|female|28.21673004870742|
|  male|30.50582424304207|
+-----+-----+
```

```
# =====
# Total Fare Collected by Passenger Class
# =====

df.groupBy("Pclass") \
  .agg(
    sum("TicketFare").alias("Total Fare")
  ) \
  .show()
```

```
+-----+-----+
|Pclass|      Total Fare|
+-----+-----+
|     1|18177.412499999995|
|     3| 6714.695100000003|
|     2|       3801.8417|
+-----+-----+
```

✓ HAVING Condition

Display only Passenger Classes where the total fare collected is greater than 5000.

Spark uses `filter()` after `groupBy()` to achieve a HAVING-like result.

```
# =====
# HAVING Condition
# =====

df.groupBy("Pclass") \
  .agg(
    sum("TicketFare").alias("Total Fare")
  ) \
  .filter(col("Total Fare") > 5000) \
  .show()
```

```
+-----+-----+
|Pclass|      Total Fare|
+-----+-----+
```

```
|      1|18177.412499999995|
|      3| 6714.695100000003|
+-----+-----+
```

✓ Sorting Data

Sorting arranges records in ascending or descending order.

Functions used:

- `orderBy()`
- `desc()`

```
# =====
# Sort by Age (Ascending)
# =====

df.orderBy("Age").show(10)
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      804 |      1 |      3 | Thomas, Master. A... | male | 0.42 |      0 |      1 | 262 |
|      756 |      1 |      2 | Hamalainen, Maste... | male | 0.67 |      1 |      1 | 25064 |
|      645 |      1 |      3 | Baclini, Miss. Eu... | female | 0.75 |      2 |      1 | 266 |
|      470 |      1 |      3 | Baclini, Miss. He... | female | 0.75 |      2 |      1 | 266 |
|      832 |      1 |      2 | Richards, Master.... | male | 0.83 |      1 |      1 | 2910 |
|      79 |      1 |      2 | Caldwell, Master.... | male | 0.83 |      0 |      2 | 24873 |
|      306 |      1 |      1 | Allison, Master. ... | male | 0.92 |      1 |      2 | 11378 |
|      184 |      1 |      2 | Becker, Master. R... | male | 1.0 |      2 |      1 | 23013 |
|      173 |      1 |      3 | Johnson, Miss. El... | female | 1.0 |      1 |      1 | 34774 |
|      165 |      0 |      3 | Panula, Master. E... | male | 1.0 |      4 |      1 | 310129 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

only showing top 10 rows

```
# =====
# Sort by Ticket Fare (Descending)
# =====

df.orderBy(
    desc("TicketFare")
).show(10)
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      680 |      1 |      1 | Cardeza, Mr. Thom... | male | 36.0 |      0 |      1 | PC 177 |
|      738 |      1 |      1 | Lesurer, Mr. Gust... | male | 35.0 |      0 |      0 | PC 177 |
|      259 |      1 |      1 | Ward, Miss. Anna | female | 35.0 |      0 |      0 | PC 177 |
|      342 |      1 |      1 | Fortune, Miss. Al... | female | 24.0 |      3 |      2 | 199 |
|      89 |      1 |      1 | Fortune, Miss. Ma... | female | 23.0 |      3 |      2 | 199 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

28	0	1	Fortune, Mr. Char...	male	19.0	3	2	199
439	0	1	Fortune, Mr. Mark	male	64.0	1	4	199
743	1	1	"Ryerson, Miss. S...	female	21.0	2	2	PC 176
312	1	1	Ryerson, Miss. Em...	female	18.0	2	2	PC 176
119	0	1	Baxter, Mr. Quigg...	male	24.0	0	1	PC 175

only showing top 10 rows

```
# =====
# Sort by Passenger Class and Age
# =====

df.orderBy(
    "Pclass",
    "Age"
).show(10)
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
306	1	1	Allison, Master. ...	male	0.92	1	2	1137
298	0	1	Allison, Miss. He...	female	2.0	1	2	1137
446	1	1	Dodge, Master. Wa...	male	4.0	0	2	336
803	1	1	Carter, Master. W...	male	11.0	1	2	1137
436	1	1	Carter, Miss. Luc...	female	14.0	1	2	1137
690	1	1	Madill, Miss. Geo...	female	15.0	0	1	241
330	1	1	Hippach, Miss. Je...	female	16.0	0	1	1113
505	1	1	Maioni, Miss. Rob...	female	16.0	0	0	1101
854	1	1	Lines, Miss. Mary...	female	16.0	0	1	PC 175
551	1	1	Thayer, Mr. John ...	male	17.0	0	2	174

only showing top 10 rows

```
# =====
# Top 10 Highest Fare Passengers
# =====

df.orderBy(
    desc("TicketFare")
).limit(10).show()
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
680	1	1	Cardeza, Mr. Thom...	male	36.0	0	1	PC 177
738	1	1	Lesurer, Mr. Gust...	male	35.0	0	0	PC 177
259	1	1	Ward, Miss. Anna	female	35.0	0	0	PC 177
342	1	1	Fortune, Miss. Al...	female	24.0	3	2	199
89	1	1	Fortune, Miss. Ma...	female	23.0	3	2	199
28	0	1	Fortune, Mr. Char...	male	19.0	3	2	199
439	0	1	Fortune, Mr. Mark	male	64.0	1	4	199
743	1	1	"Ryerson, Miss. S...	female	21.0	2	2	PC 176
312	1	1	Ryerson, Miss. Em...	female	18.0	2	2	PC 176
119	0	1	Baxter, Mr. Quigg...	male	24.0	0	1	PC 175

✓ Wide Transformations

Spark transformations are classified into two types:

1. Narrow Transformations

These transformations do not require data movement between partitions.

Examples:

- filter()
- select()
- withColumn()
- drop()

2. Wide Transformations

These transformations require moving data across partitions.

Examples:

- groupBy()
- join()
- distinct()
- repartition()

Wide transformations involve **Shuffle**, which increases network communication and execution time.

```
# =====  
# Wide Transformation Example  
# =====  
  
# Group passengers by Passenger Class  
  
wide_df = df.groupBy("Pclass").count()  
  
wide_df.show()
```

```
+-----+-----+  
|Pclass|count|  
+-----+-----+  
|      1|  216|  
|      3|  491|  
|      2|  184|
```

+-----+-----+

Shuffle Operation

The `groupBy()` operation redistributes records across partitions.

This movement of data is known as **Shuffle**.

Shuffle helps Spark perform grouping and aggregation but may increase execution time because data is exchanged between worker nodes.

✓ Complete Data Processing Pipeline

The pipeline performs:

1. Load Dataset
2. Remove Duplicates
3. Handle Missing Values
4. Rename Columns
5. Filter Data
6. Aggregate Data
7. Display Final Result

```
# =====
# Complete ETL Pipeline
# =====

pipeline_df = (
    df
    .dropDuplicates()                # Remove duplicate rows
    .fillna({"Embarked": "Unknown"}) # Fill missing Embarked values
    .filter(col("Age") >= 18)        # Keep only adult passengers
    .withColumnRenamed("TicketFare", "Fare") # Rename column
)

print("Pipeline Executed Successfully")

pipeline_df.show(10)
```

Pipeline Executed Successfully

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp
35	0	1	Meyer, Mr. Edgar ...	male	28.0	1
285	0	1	Smith, Mr. Richar...	male	29.69911764705882	0
463	0	1	Gee, Mr. Arthur H	male	47.0	0
149	0	2	Navratil, Mr. Mi...	male	36.5	0

151	0	2	Bateman, Rev. Rob...	male	51.0	0
328	1	2	Ball, Mrs. (Ada E...	female	36.0	0
530	0	2	Hocking, Mr. Rich...	male	23.0	2
689	0	3	Fischer, Mr. Eber...	male	18.0	0
710	1	3	"Moubarek, Master...	male	29.69911764705882	1
767	0	1	Brewe, Dr. Arthur...	male	29.69911764705882	0

only showing top 10 rows

✓ Aggregation on Processed Data

Calculate the total fare collected from passengers grouped by passenger class.

```
# =====
# Aggregate Processed Data
# =====

pipeline_result = pipeline_df.groupBy("Pclass") \
    .agg(
        count("*").alias("Passenger Count"),
        avg("Fare").alias("Average Fare"),
        sum("Fare").alias("Total Fare")
    )

pipeline_result.show()
```

Pclass	Passenger Count	Average Fare	Total Fare
1	204	82.74732450980389	16880.454199999993
3	413	12.302318159806308	5080.857400000005
2	161	19.904891304347828	3204.6875000000005

✓ Save Processed Data

The processed DataFrame is saved as a CSV file.

```
# =====
# Save Processed Dataset
# =====

pipeline_result.write.mode("overwrite") \
    .option("header", True) \
    .csv("Spark_Output")

print("Processed Data Saved Successfully.")
```

Processed Data Saved Successfully.

✓ Download Saved Files (Google Colab)

The output is stored inside the `Spark_Output` folder.

```
# =====
# Display Output Folder
# =====

!ls Spark_Output
```

```
part-00000-81c6e123-6f7c-41b3-b9a5-6d1c00b3d372-c000.csv  _SUCCESS
```

✓ Display Final Processed Dataset

```
# =====
# Display Final Dataset
# =====

pipeline_result.show(truncate=False)
```

Pclass	Passenger Count	Average Fare	Total Fare
1	204	82.74732450980389	16880.454199999993
3	413	12.302318159806308	5080.857400000005
2	161	19.904891304347828	3204.6875000000005

Observations

- The Titanic dataset was successfully loaded into Spark.
- Duplicate records were removed using `dropDuplicates()`.
- Missing values in **Age**, **Cabin**, and **Embarked** were handled.
- Columns were renamed and transformed using `withColumnRenamed()` and `withColumn()`.
- Filtering operations were applied to retrieve meaningful subsets of data.
- Aggregation functions such as `count()`, `sum()`, `avg()`, `min()`, and `max()` were used to summarize the dataset.
- `groupBy()` was used to analyze passenger distribution and fare statistics.
- Sorting operations helped identify passengers with the highest fares.

- A complete ETL pipeline was created by combining cleaning, transformation, filtering, and aggregation.
- The processed results were successfully saved as a CSV file.

✓ Conclusion

This assignment demonstrated the core features of Apache Spark DataFrames.

Key concepts implemented include:

- Spark Session creation
- Loading CSV files
- Exploring DataFrames
- Data Cleaning
- Handling Missing Values
- Removing Duplicate Records
- Data Transformation
- Schema Modification
- Filtering
- Aggregation
- GroupBy Operations
- Sorting
- Wide Transformations
- Shuffle Concept
- Building an End-to-End Data Processing Pipeline
- Saving Processed Data

Spark's in-memory processing and optimized execution make it significantly faster and more efficient than traditional MapReduce for large-scale data analysis.

Double-click (or enter) to edit